



Security Audit

Noon Capital Updates V2 (Governance Token)



Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	26
Conclusion	27
Our Methodology	28
Disclaimers	30
About Hashlock	31

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Noon Capital team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Noon Capital is a DeFi governance protocol built on the Ethereum Network that allows users to stake NOON tokens and participate in decentralized governance through a veToken system. The protocol features time-locked staking with voting power multipliers, VIP staking tiers with enhanced rewards, and a comprehensive vesting mechanism for long-term token distribution.

In simple terms, users can stake NOON tokens to gain voting power in governance decisions while earning additional rewards through the vesting system, creating aligned incentives for long-term participation in the protocol.

Project Name: Noon Capital

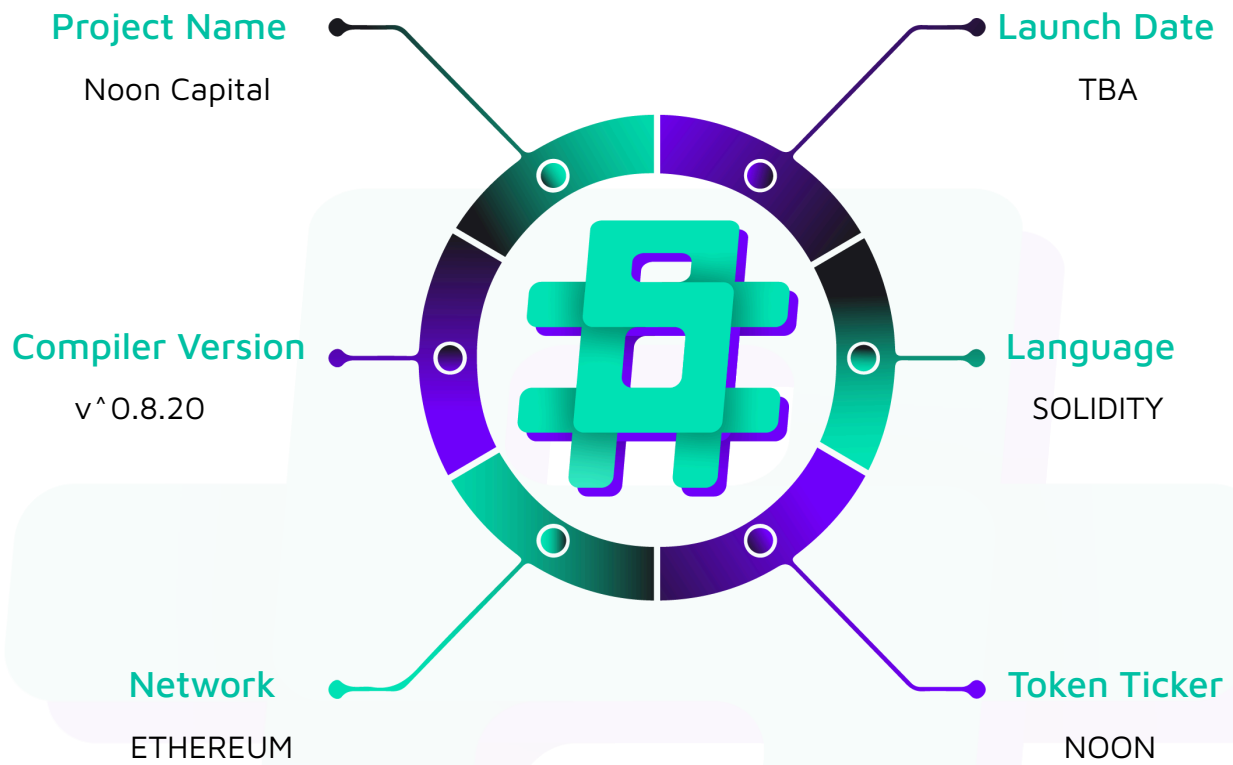
Project Type: Governance Token

Compiler Version: ^0.8.20

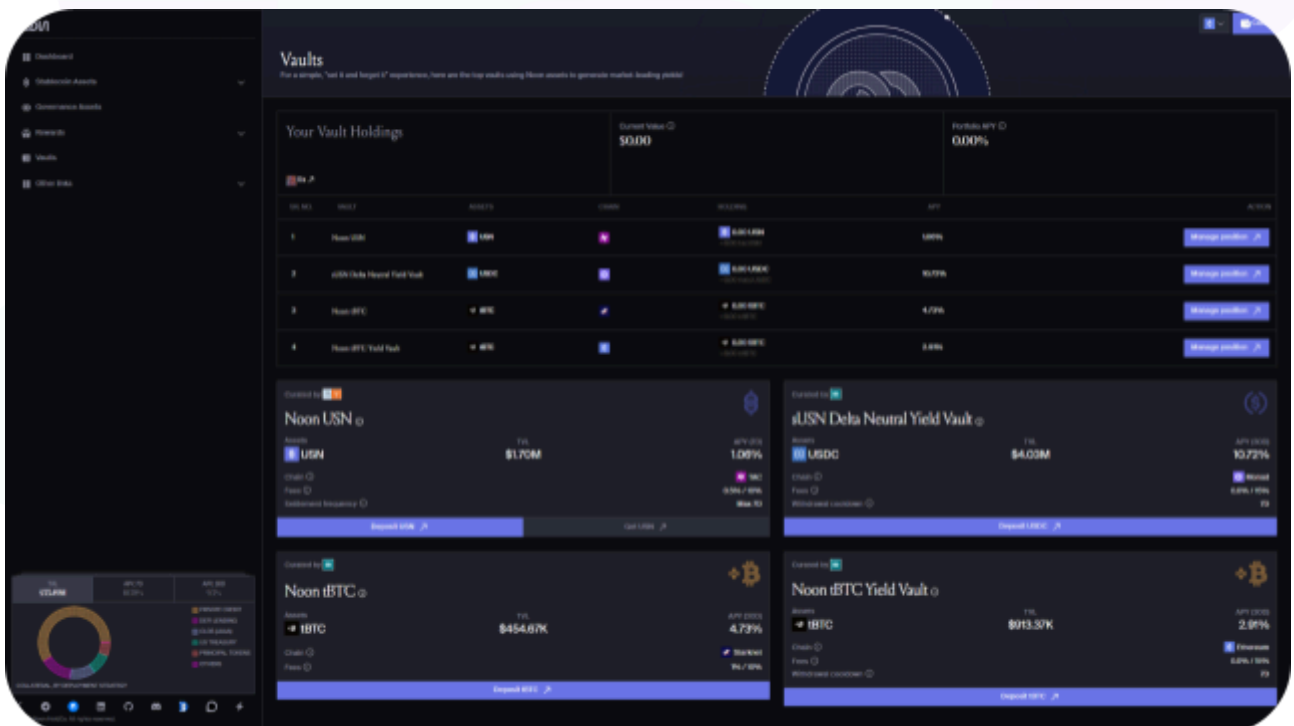
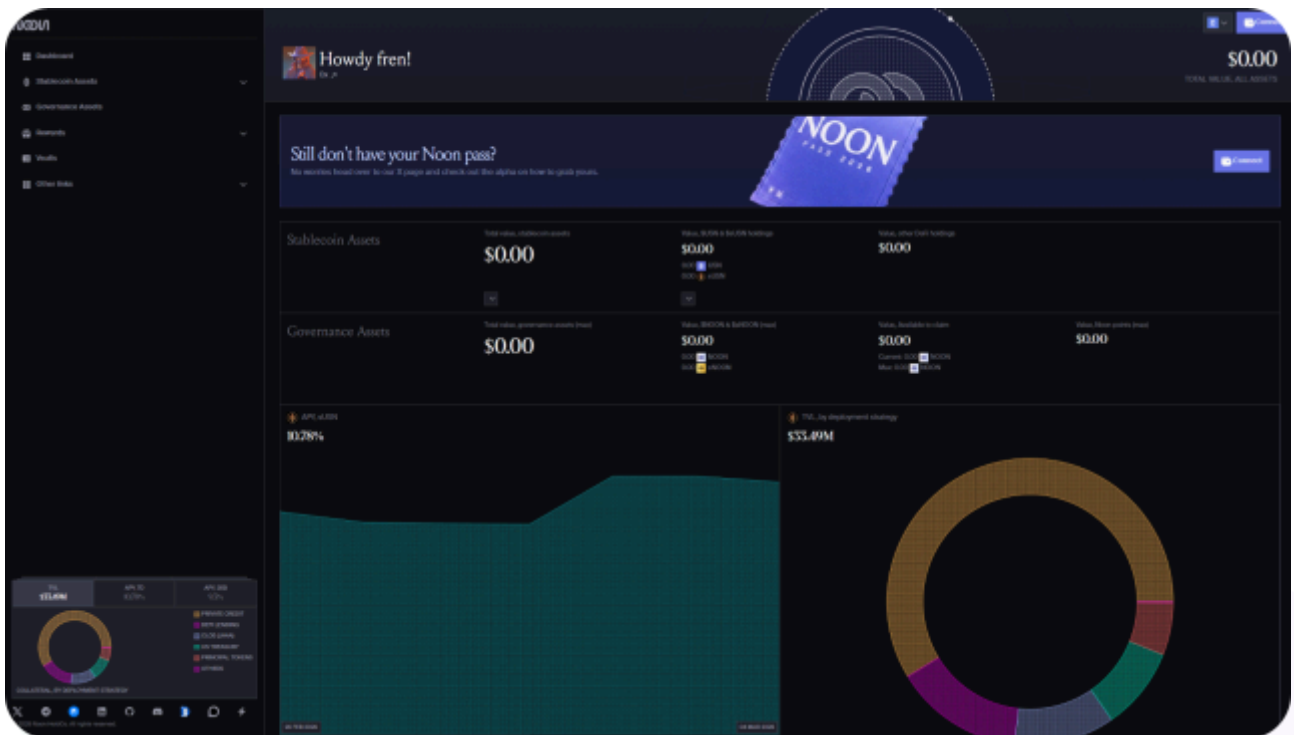
Website: <https://noon.capital/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Noon Capital project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Noon Capital Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	March, 2026
Contract 1	stakeNOON.sol
Contract 2	stakeNOONVesting.sol
Audited GitHub Commit Hash	3afde1de153f733ce82d12086b3d84f89daff3be
Fix Review GitHub Commit Hash	75b039505786268061b4fa15cf108ab745e8cf71

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 Medium severity vulnerabilities

4 Low severity vulnerabilities

4 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
stakeNOON.sol - Allows the owner to: - Update max state limits - Toggle NFT transferability - Set merkle root and fund claim pool - Set vesting contract address - Set VIP unlocking period - Set VIP stake percentage - Allows users to: - Create stake - Update stake - Withdraw - Claim and stake from merkle allocation	Contract achieves this functionality.
stakeNOONVesting.sol - Allows the owner to: - Add vesting allocation tokens - Allows staking contract to: - Create vesting schedules - Execute vesting claims for users	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Noon Capital project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Noon Capital project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] stakeNOON#setVipUnlockingPeriod, startVIPUnstake, withdrawVip -
Unlock period can be changed retroactively for active unstakes

Description

VIP unstake timing is not locked when the user initiates unstake.

Vulnerability Details

startVIPUnstake records start time only. withdrawVip checks a live global:

```
if (block.timestamp < vipUnlockStartTime[tokenId] + vipUnlockingPeriod) revert  
UnlockPeriodNotCompleted();
```

withdrawVip reads vipUnlockingPeriod at the time of withdrawal, not at the time startVIPUnstake was called. This means the owner can update it after a user has already started their unlock, extending the wait or even setting it to `type(uint256).max` to prevent withdrawal entirely for users already in the unlock window.

Impact

Users can be forced to wait longer than the terms that existed when they started unstaking. In bigger values, withdrawals can become practically blocked.

Recommendation

Snapshot unlock period per token when unstake starts and use that snapshot in withdrawVip and claimVesting.

Status

Resolved

[M-02] stakeNOON#withdrawVip - VIP withdrawal suppresses vesting claim failures and still finalizes stake

Description

The `try/catch` loop in `withdrawVip` silently ignores failed vesting claims and then calls `_finalizeVipStake` regardless, which burns the NFT and clears stake state. If any claim fails, for example because the vesting contract is insolvent, the user loses both the NFT and any unclaimed vesting amounts with no way to retry.

Vulnerability Details

The function loops with `try/catch`, emits on failure, and then finalizes regardless:

```
try vestingContract.claimVesting(msg.sender, i - 1) {  
    // Claim succeeded  
} catch {  
    // Claim failed - emit event and continue  
    emit VestingClaimFailed(tokenId);  
}...  
_finalizeVipStake(tokenId, msg.sender);
```

If any vesting claim fails, cleanup still proceeds and the retry path is lost.

Impact

Users can lose the ability to recover failed vesting amounts after finalization.

Recommendation

Do not allow finalization if any vesting claim fails. Either revert on first failure or enforce a two-step flow where finalization is allowed only when no vesting schedules remain.

Status

Resolved

[M-03] stakeNOONVesting#createVestingSchedule - Incorrect liability accounting causes vesting insolvency

Description

The vesting contract checks available capacity against `amount`, but each schedule commits `amount * VESTING_MULTIPLIER` as the payout liability.

Vulnerability Details

```
require(amount <= totalVestingAllocation - totalClaimed, InsufficientVestingAllocation());  
  
...  
  
totalAmount: amount * VESTING_MULTIPLIER
```

A new schedule can pass the check even when the full vesting obligation is not fundable. The mismatch compounds as more schedules are created.

Impact

The vesting pool can be over committed and become insolvent. Users who claim earlier are paid first, while later users may be unable to claim vested tokens.

Recommendation

Track active obligations with `totalScheduled` and enforce solvency using the full liability being added. The check should validate:

```
totalClaimed + totalScheduled + (amount * VESTING_MULTIPLIER) <= totalVestingAllocation.
```

Update `totalScheduled` on both schedule creation and schedule removal.

Status

Acknowledged

[M-04] stakeNOON#setMerkleRoot - Merkle root rotation breaks round accounting and leaves prior round balances unclaimable

Description

Merkle root updates replace proof state, but accounting remains global and shared across rounds.

Vulnerability Details

setMerkleRoot overwrites merkleRoot while incrementing cumulative balance accounting:

```
function setMerkleRoot(bytes32 _merkleRoot, uint256 totalAmount) external onlyOwner {  
    if (totalAmount == 0) revert AmountMustBeGreaterThanZero();  
    require(noon.transferFrom(msg.sender, address(this), totalAmount), TokenTransferFailed());  
    merkleRoot = _merkleRoot;  
    totalClaimableAmount += totalAmount;  
    emit MerkleRootUpdated(_merkleRoot, totalAmount);}
```

Once the root is replaced, users who did not claim under the previous root can no longer provide a valid proof. At the same time, claimedAmounts[user] is global, so prior round claims affect later rounds.

Impact

Unclaimed balances from previous rounds can become unclaimable. Cross round accounting can also produce unintended claim behavior.

Recommendation

Use per-round accounting for deposits, user claims, and claimed totals. Before replacing a root, include a defined settlement path for the previous round (claim window and remainder handling).

Status

Acknowledged

Low

[L-01] stakeNOONVesting#getVestingSchedulesForStake - Returns all user schedules instead of stake-filtered schedules

Description

The function resolves owner from `stakeId` and returns `vestingSchedules[user]` without filtering by `stakeId`.

Recommendation

Return only schedules where `schedule.stakeId == stakeId`.

Status

Resolved

[L-02] stakeNOON#claimVesting - Missing explicit schedule to token binding check

Description

`claimVesting` uses a schedule index from returned schedules but does not explicitly enforce `schedules[scheduleId].stakeId == tokenId`.

Recommendation

Add a direct binding check before changing `stake.amount`, `totalStaked`, and transfer logic.

Status

Resolved

[L-03] **stakeNOON#withdrawWithReward, claimVesting, claimWithdrawalReward, _finalizeVipStake** - Unchecked ERC20 transfer return handling

Description

`withdrawWithReward`, `_finalizeVipStake`, and `claimVesting` in `stakeNOON` all call `noon.transfer` without checking the return value. State is already updated before the transfer in most of these paths, so a token that returns `false` instead of reverting would silently fail to pay the user while the contract moves on as if it succeeded. Other paths like `setMerkleRoot` and `addVestingAllocation` use `require`, the inconsistency can make this a risk if token behavior changes.

Recommendation

Use `require` on all `transfer/transferFrom` calls or switch to `SafeERC20.safeTransfer` across both contracts.

Status

Resolved

[L-04] stakeNOON#_finalizeVipStake, claimWithdrawalReward - VIP users cannot claim later withdrawal rewards

Description

`_finalizeVipStake` never sets `withdrawnStakeOwners[tokenId]`, but `claimWithdrawalReward` requires that mapping to be set to the caller's address. VIP users who withdraw will always hit the `NotOwner` revert when trying to claim a merkle-based reward for their withdrawn stake, even if a valid proof exists for them.

```
if (withdrawnStakeOwners[tokenId] != msg.sender) revert NotOwner();
```

Recommendation

Add `withdrawnStakeOwners[tokenId] = user` in `_finalizeVipStake` before the stake data is deleted, so VIP exits leave the same ownership trail that normal withdrawals do.

Status

Resolved

QA

[Q-01] stakeNOON#updateStake - Unused local variable

Description

`totalDuration` is computed in both branches but never used.

Recommendation

Remove the unused variable and assignments.

Status

Resolved

[Q-02] stakeNOON - Unused `immediateClaimableAmount` **mapping****Description**

`immediateClaimableAmount` is declared with a public getter but never written or read.

Recommendation

Remove unused mapping to reduce storage.

Status

Resolved

[Q-03] stakeNOON#setVestingContract - Missing event emission for admin change

Description

Updating vesting contract address has no dedicated event, reducing traceability for governance and monitoring tools.

Recommendation

Emit an event in `setVestingContract`.

Status

Resolved

[Q-04] IStakeNOON, IStakeNOONVesting - Event signature mismatch across interfaces

Description

VestingScheduleCreated is declared in IStakeNOON with 3 parameters (user, tokenId, amount) but the vesting contract emits a 6 parameter version (user, scheduleId, amount, startTime, endTime, stakeId). Any off-chain consumer using the IStakeNOON ABI to decode this event will get bad data.

Recommendation

Remove or update the stale declaration in IStakeNOON to match what the vesting contract actually emits.

Status

Resolved

Centralisation

The Noon Capital project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Noon Capital project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.